

Subagents & Teams Guide

Day 3 Handout - Single Session vs Subagents vs Nested vs Teams (July 2026)

The Four Architectures

	Single session	Subagents	Nested subagents	Agent Teams
Coordination	you	main agent	coordinator agents	team runtime (peer mailboxes)
Context	one window	isolated per agent	isolated per level	isolated + git worktrees
Communication	-	report to caller	report up the tree	peer-to-peer
Cost	lowest	per-agent multiplier	multiplies per level	highest
Status	stable	stable	stable (v2.1.172+)	EXPERIMENTAL - disabled by default (verify at delivery - fast-moving)

Subagents - the Foundation

Markdown files in `.claude/agents/` with YAML frontmatter:

```
---
name: security-auditor
description: Audit code for security vulnerabilities. Use when asked to
"check security", "audit for vulnerabilities", or "review auth code".
tools: Read, Grep, Glob
---
# Security Auditor
Check for: hardcoded secrets, SQL injection, unvalidated input, eval().
Output: CRITICAL / WARNING / INFO with file:line references.
```

Key facts:

- **Isolation:** own context window, own tool set, optionally own model. Can't see your conversation.
- **Least privilege:** the `tools` field is enforcement - give reviewers read-only tools and they cannot "helpfully" edit code.
- **Background + streaming (week 27):** subagents run in the background while the main agent keeps working; their text streams live.
- **Files are the API:** agents hand off via files (`reports/security.md`), not shared memory.
- Invoke by description match ("audit this for security"), by name, or programmatically.

Nested Subagents (v2.1.172, June 2026)

Subagents can spawn their own subagents - up to ~5 levels deep (verify at delivery - fast-moving).

Shape: you -> coordinator -> workers. The coordinator holds MERGE logic; workers hold DETAIL. Your main context receives one synthesized report instead of N raw dumps.

When to nest: fan-out analysis (inventory + gaps + risks merged into one report), research trees, anything where intermediate output would bloat your window.

When NOT to nest: simple sequential work - every level multiplies token cost and adds latency.

Classic failure: the coordinator does the work itself. Prompt it explicitly: *"Do NOT analyze directly - delegate to subagents and only merge their outputs."*

The Grader Pattern (Performance Outcomes)

Work isn't done until a SEPARATE agent scores it against an explicit rubric:

1. Write `rubric.md` - 5 binary-ish criteria
2. A read-only **grader** subagent scores each criterion 0-2 with evidence (file:line)
3. Verdict: **TOTAL:** `n/10`, PASS 8 or REWORK
4. Builder fixes ONLY failing criteria; grader re-scores

Why separate: the grader's context contains the rubric and the code - not the builder's rationalizations. Same reason human code review isn't done by the author. Scales to fan-out: dynamic workflows can grade hundreds of parallel workers against one rubric.

Agent Teams - EXPERIMENTAL

A built-in multi-agent orchestrator: a lead spawns teammates with roles; teammates work in their own git worktrees and message each other directly (mailbox), then the lead merges.

- **Disabled by default; flag name and availability vary by version and plan (verify at delivery - fast-moving).**
- Terminal/tmux gives split-pane visibility; VS Code shows progress reports instead.
- **Fallback that always works:** run the same role breakdown as parallel subagents coordinated by your main agent. You lose peer-to-peer messaging; you keep isolation and parallelism.

Decision Guide

Task	Architecture
Quick fix, one file	Single session
Focused analysis you want out of your context	One subagent
Independent parallel tasks (build + tests + docs)	Parallel subagents
Fan-out analysis merged into one report	Nested (coordinator + workers)
Quality gate on any of the above	+ grader subagent
Multi-file feature needing peer coordination, isolated branches	Agent team (if flag available - else nested/parallel fallback)

Cost rule: each agent ~ its own session cost (loads CLAUDE.md, reads files, reasons independently). Two subagents ~ 2x doing it yourself. Pay for isolation or parallelism - never for ceremony.

Ops Notes

- `/agents` - list definitions; `/context` - watch multi-agent token burn
- SubagentStart/SubagentStop/TeammateIdle hook events give you telemetry (see Hooks handout)
- Worktree isolation got fixes in v2.1.210 - update before heavy team use
- Subagent-not-found: definitions must be in the PROJECT's `.claude/agents/`; `/clear` to reload

(c) 2026 AIA Copilot - Claude Code Training